# HDOC Day-59

**Longest Substring Without Repeating Characters in C**

## Problem Statement

Read a string s, pass its address to longest_unique_substring(), and display the length of its longest contiguous substring containing no repeated characters. Treat uppercase and lowercase as different characters. Display 0 for an empty string.

## Algorithm

1. Start and declare a character array s and an array last[256] to store last-seen positions.
2. Read a complete line using fgets(); remove its trailing newline if present.
3. Pass the address of s to longest_unique_substring(const char *s).
4. Initialize every entry in last[] to -1, left = 0 and best = 0.
5. Scan characters from left to right using index i. Convert s[i] to unsigned char to use as an array index.
6. If the character was last seen at or after left, set left = last[character] + 1.
7. Update last[character] = i. Calculate the current unique-window length i - left + 1 and update best if larger.
8. Display best. If s is empty, the loop is skipped and best remains 0. Stop.

## Test Case Table

| Test Case | Input String | Expected Length | Purpose |
|---|---|---|---|
| 1 | abcabcbb | 3 | Professor example: repeated letters |
| 2 | bbbbb | 1 | Professor example: all identical |
| 3 | pwwkew | 3 | Professor example: wke |
| 4 | [press Enter without typing] | 0 | Empty input |
| 5 | abcdef | 6 | All unique characters |
| 6 | aA | 2 | Uppercase and lowercase differ |
| 7 | abba | 2 | Repeated middle letters |
| 8 | a b a | 3 | Spaces count as characters |
| 9 | dvdf | 3 | Longest substring vdf |
| 10 | x | 1 | Single character |

Note: For the empty-string test case, simply press Enter. A space is treated as a character. The program uses zero-based character positions internally.

## C Program

```
#include <stdio.h>
#include <string.h>

#define MAX 1000

void longest_unique_substring(const char *s)
```

```
{
    int last[256];
    int i, left = 0, best = 0;
    for (i = 0; i < 256; i++) last[i] = -1;
    for (i = 0; s[i] != '\0'; i++) {
        unsigned char ch = (unsigned char)s[i];
        if (last[ch] >= left) left = last[ch] + 1;
        last[ch] = i;
        if (i - left + 1 > best) best = i - left + 1;
    }
    printf("Length of longest substring: %d\n", best);
}

int main(void)
{
    char s[MAX];
    size_t length;
    int ch;
    printf("Enter a string: ");
    if (fgets(s, sizeof(s), stdin) == NULL) {
        printf("Input error.\n");
        return 1;
    }
    length = strlen(s);
    if (length > 0 && s[length - 1] == '\n') s[length - 1] = '\0';
    else if (length == MAX - 1) {
        while ((ch = getchar()) != '\n' && ch != EOF) { }
    }
    longest_unique_substring(s);
    return 0;
}
```

## Compilation and Execution

```
gcc hdoc59.c -o hdoc59
./hdoc59
```

## Screenshot - Compilation and Execution

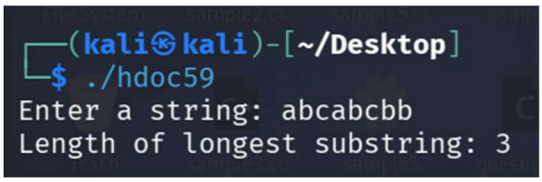



## Test Case 1

Input: abcabcbb

Expected output: Length of longest substring: 3

Professor example: repeated letters

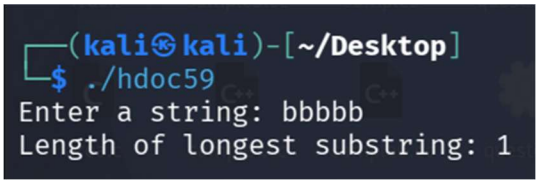



## Test Case 2

Input: bbbbb

Expected output: Length of longest substring: 1

Professor example: all identical

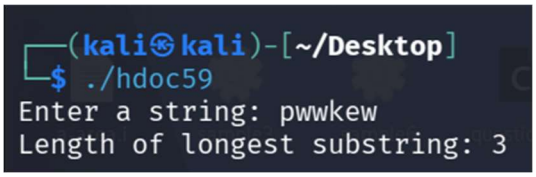



## Test Case 3

Input: pwwkew

Expected output: Length of longest substring: 3

Professor example: wke

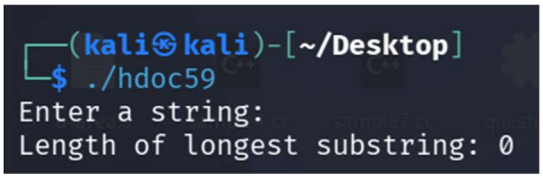


## Test Case 4

Input: [press Enter without typing]

Expected output: Length of longest substring: 0

Empty input

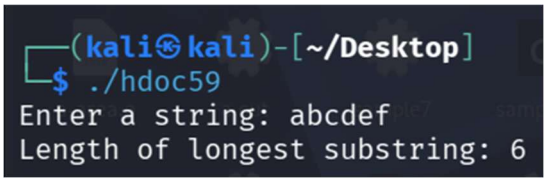



## Test Case 5

Input: abcdef

Expected output: Length of longest substring: 6

All unique characters

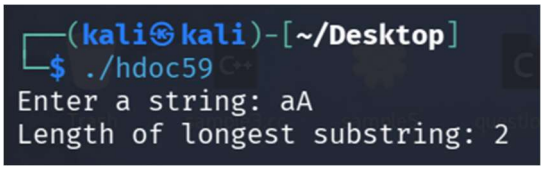



## Test Case 6

Input: aA

Expected output: Length of longest substring: 2

Uppercase and lowercase differ

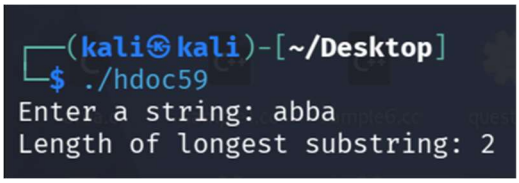



## Test Case 7

Input: abba

Expected output: Length of longest substring: 2

Repeated middle letters

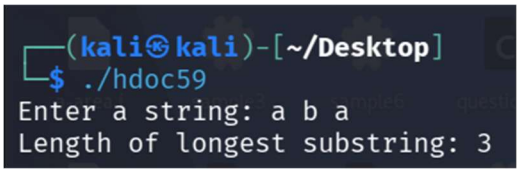


## Test Case 8

Input: a b a

Expected output: Length of longest substring: 3

Spaces count as characters

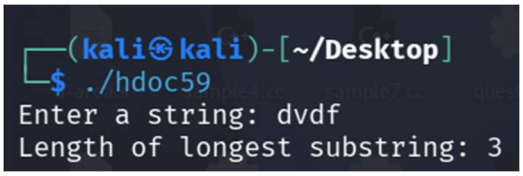



## Test Case 9

Input: dvdf

Expected output: Length of longest substring: 3

Longest substring vdf




## Test Case 10

Input: x

Expected output: Length of longest substring: 1

Single character

